

Data Structures and Algorithms Lab 2: Sorting & Linked Lists Report

Student: Cao Bảo Khương (ITDSIU22176)

Part A: Sorting Observations

Both MergeSort and QuickSort achieved $O(N \log N)$ runtime. Our QuickSort leverages a **Median-of-three** pivot selection strategy. This limits the worst-case scenario drastically compared to choosing the first or last element, ensuring solid performance even on roughly sorted datasets or immediately reverse datasets.

Complexity Table: - Merge Sort: Best/Avg/Worst Time = $O(N \log N)$, Space = $O(N)$. - Quick Sort: Best/Avg Time = $O(N \log N)$, Worst Time = $O(N^2)$ (mitigated gracefully by Median-of-Three), Space = $O(\log N)$ stack max scale.

Benchmark Output Excerpt (Exercise 2 - N=1000): - **Random Dataset:** Multi-way recursion depth maintained within log boundaries. Swaps in QuickSort were observed to be inherently minimal than Merge Sort's array copying. - **Already Sorted:** Quick Sort performance remained comparably strong. The pivot essentially scales without trailing standard partitions.

Part B: Linked List Fundamentals

Full structural algorithms implementations have been verified: - Standard singly and doubly structures efficiently support $O(1)$ insertions at extremes (with active tail variable management implemented). - Iterative reversing for Singly Linked list models proved superior overall to recursively reversing logic due to mitigating massive $O(N)$ stack consumption in extremely large datasets. - The CycleTools implementations satisfy space-restrictive bounds utilizing Floyd's Two-Pointers tracking, vastly outperforming simpler HashSet $O(N)$ memory scaling alternatives on cycle start detections.

Part C: Exercises Justifications

Exercise 3 (Streaming Merge): Achieved natively in $O(N+M)$ time simply by reconnecting pointers and avoiding entirely array rebuild overhead heuristics. **Exercise 6 (Stable Partition):** Using discrete placeholder lessHead and greaterHead link lines is significantly faster and conservatively uses less memory than tracking nested swapping behaviors inside lists. Time = $O(N)$, Space = $O(1)$. **Exercise 7 (LL Sorting):** Implemented Top-Down Merge Sort cleanly bounded at $O(N \log N)$ complexity, fundamentally outperforming generic sequential scans or standard slow selection sort implementations for Singly Lists boundaries since standard Quicksort limits pivot access cleanly. **Exercise 9 (LRU Cache):** The hybrid combination of a robust Doubly Linked List tracking history lifecycle paired

interactively with a standard Java HashMap achieves strict $O(1)$ direct operation bounds structurally mandated by massive system scale caching constraints.